

```

Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
pk      | integer |           | not null |
tag     | text    |           |          |
parent  | integer |           |          |

Indexes:
    "table_b_pk" PRIMARY KEY, btree (pk)
Inherits: table_a

```

Here, we can see that `table_b` is a child table of `table_a`.

4. Now let's see how these two tables behave if we insert, modify, or delete data. For example, let's make some inserts as follows:

```

forumdb=> insert into table_a (pk,tag,parent) values (1,'Operating
Systems',0);
INSERT 0 1

forumdb=> insert into table_b (pk,tag,parent) values (2,'Linux',0);
INSERT 0 1

```

5. Let's see how our data reacts if we execute the select command:

```

forumdb=> select * from table_b ;
pk | tag | parent
----+-----+-----
  2 | Linux |      0
(1 row)

```

We can see that `table_b` has one record.

6. Now we execute the following command:

```

forumdb=> select * from table_a ;
pk | tag | parent

```

```

----+-----+-----
  1 | Operating Systems |      0
  2 | Linux           |      0
(2 rows)

```

It seems that `table_a` has two records. This happens because this table inherits the other table's attributes. If we execute a `SELECT` command on a parent table, we will see all the records that belong to the parent table and all the records that belong to the child table.

7. If we want to see all the records that belong to `table_a` only, we have to use the `ONLY` clause, as seen here:

```

forumdb=> select * from only table_a ;
pk |      tag      | parent
----+-----+-----
  1 | Operating Systems |      0
(1 row)

```

8. Let's see what happens if we `UPDATE` some records, for example, if we execute the following:

```

forumdb=> update table_a set tag='BSD Unix' where pk=2;
UPDATE 1

```

We performed an update operation on `table_a`, but this update was physically done on `table_b` by means of the inheritance of the tables, as we can see here:

```

forumdb=> select * from table_b;
pk |  tag  | parent
----+-----+-----
  2 | BSD Unix |      0
(1 row)

```

9. The same happens if we use a `delete` statement as follows:

```

forumdb=> delete from table_a where pk=2;
DELETE 1

```

Here, again, the delete operation performed on `table_a` has its effect on `table_b`; as we can see here, `table_a` will have these records:

```
forumdb=> select * from table_a;
pk |      tag      | parent
----+-----+-----
  1 | Operating Systems |      0
(1 row)
```

And `table_b` will now have no records:

```
forumdb=> select * from table_b;
pk | tag | parent
----+----+-----
(0 rows)
```

In PostgreSQL, inheritance propagates the operations performed on the parent table to the child tables.

Dropping tables

To conclude the topic of inheritance, we need to address how to delete tables. If we want to delete a child table, for example, to drop `table_b`, we have to run the following statement:

```
forumdb=> drop table table_b;
DROP TABLE
```

If we want to drop a parent table and all its linked child tables, we have to run the following:

```
forumdb=> drop table table_a cascade;
```

While inheritance has been used and still can be used to implement table partitioning, since version 10 declarative partitioning has become the preferred method. We cover declarative partitioning in the next section.

Exploring declarative partitioning

In this section, we will talk about declarative partitioning. It has been available in PostgreSQL since version 10, but its performance has increased in newer versions. We will now look at an example of partitioning by range and an example of partitioning by list.

List partitioning

In the first example of declarative partitioning, we will use the same example that we looked at when we introduced partitioning using inheritance. We will see that things become much simpler using the declarative partitioning method:

1. Now let's create our parent table:

```
forumbd=> CREATE TABLE part_tags (  
    pk SERIAL NOT NULL ,  
  
    level INTEGER NOT NULL DEFAULT 0,  
    tag VARCHAR (255) NOT NULL,  
    primary key (pk,level)  
)  
PARTITION BY LIST (level);
```

As we can see from the preceding example, we have to define what kind of partitioning we want to apply. In this case, it is LIST PARTITIONING. Another important thing to note is that the field used to partition the data must be part of the primary key.

2. Next, let's define the child tables:

```
forumbd=> CREATE TABLE part_tags_level_0 PARTITION OF part_tags FOR  
VALUES IN (0);  
CREATE TABLE part_tags_level_1 PARTITION OF part_tags FOR VALUES IN  
(1);  
CREATE TABLE part_tags_level_2 PARTITION OF part_tags FOR VALUES IN  
(2);  
CREATE TABLE part_tags_level_3 PARTITION OF part_tags FOR VALUES IN  
(3);  
CREATE TABLE  
CREATE TABLE  
CREATE TABLE  
CREATE TABLE
```

With these SQL statements, we are defining the fact that all records with a level value equal to 0 will be stored in the `part_tags_level_0` table, all the records with a level value equal to 1 will be stored in the `part_tags_level_1` table, and so on.

3. Now, let's define the indexes for the parent table. These indexes will automatically be propagated to child tables. We can do this using the following simple statement:

```
forumbd=> CREATE INDEX on part_tags (tag);
CREATE INDEX
```

4. As shown here, our partition procedure is finished.

For the parent tables, we have the following:

```
forumbd=> \d part_tags;

                                Partitioned table "forum.part_tags"
  Column |          Type          | Collation | Nullable |
Default

-----+-----+-----+-----+-----
pk       | integer                |           | not null |
nextval('part_tags_pk_seq'::regclass)
level    | integer                |           | not null | 0
tag      | character varying(255) |           | not null |

Partition key: LIST (level)
Indexes:
    "part_tags_pkey" PRIMARY KEY, btree (pk, level)
    "part_tags_tag_idx" btree (tag)
Number of partitions: 4 (Use \d+ to list them.)
```

For the child tables, we have the following:

```
forumbd=> \d part_tags_level_0;

                                Table "forum.part_tags_level_0"
  Column |          Type          | Collation | Nullable |
Default

-----+-----+-----+-----+-----
pk       | integer                |           | not null |
nextval('part_tags_pk_seq'::regclass)
level    | integer                |           | not null | 0
tag      | character varying(255) |           | not null |
```

```

Partition of: part_tags FOR VALUES IN (0)
Indexes:
    "part_tags_level_0_pkey" PRIMARY KEY, btree (pk, level)
    "part_tags_level_0_tag_idx" btree (tag)

```

5. Let's now perform some INSERT operations:

```

forumdb=> insert into part_tags (tag,level) values ('Operating
System',0);
INSERT 0 1
forumdb=> insert into part_tags (tag,level) values ('Linux',1);
INSERT 0 1
forumdb=> insert into part_tags (tag,level) values ('BSD Unix',1);
INSERT 0 1
forumdb=> insert into part_tags (tag,level) values ('DOS',1);
INSERT 0 1
forumdb=> insert into part_tags (tag,level) values ('Windows',2);
INSERT 0 1

```

6. Finally, let's check whether everything is okay:

```

forumdb=> select * from part_tags;
 pk | level |      tag
-----+-----+-----
  1 |     0 | Operating System
  2 |     1 | Linux
  3 |     1 | BSD Unix
  4 |     1 | DOS
  5 |     2 | Windows
(5 rows)

forumdb=> select * from part_tags_level_0;

 pk | level |      tag
-----+-----+-----
  1 |     0 | Operating System

```

```
(1 row)

forumdb=> select * from part_tags_level_1;
pk | level | tag
----+-----+-----
 2 |      1 | Linux
 3 |      1 | BSD Unix
 4 |      1 | DOS
(3 rows)

forumdb=> select * from part_tags_level_2;
pk | level | tag
----+-----+-----
 5 |      2 | Windows
(1 row)
```

Thus, we have successfully created partitions using lists.

Range partitioning

After having seen how it is possible to partition by list in a very simple way, let's look at how to partition by range:

1. As before, let's DROP the existing `part_tags` table and its child table:

```
forumdb=> DROP TABLE IF EXISTS part_tags cascade;
DROP TABLE
```

2. Suppose that we want to have a table exactly the same as the previous one, but now we want the `part_tags` table to have an `ins_date` field where we will store the day on which the tag was added. What we want to do is partition by range on the `ins_date` field in order to put all the records entered in January 2023, February 2023, March 2023, and April 2023 into different tables. Here, we have all the statements that make this possible; they are very similar to the statements that we saw in the previous section:

```
forumdb=> CREATE TABLE part_tags (
    pk serial NOT NULL,
    ins_date date not null default now()::date,
    tag VARCHAR (255) NOT NULL,
```

```

        level INTEGER NOT NULL DEFAULT 0,
        primary key (pk,ins_date)
    )
    PARTITION BY RANGE (ins_date);
CREATE TABLE

forumdb=> CREATE TABLE part_tags_date_01_2023 PARTITION OF part_tags
FOR VALUES FROM ('2023-01-01') TO ('2023-01-31');
CREATE TABLE

forumdb=> CREATE TABLE part_tags_date_02_2023 PARTITION OF part_tags
FOR VALUES FROM ('2023-02-01') TO ('2023-02-28');
CREATE TABLE

forumdb=> CREATE TABLE part_tags_date_03_2023 PARTITION OF part_tags
FOR VALUES FROM ('2023-03-01') TO ('2023-03-31');
CREATE TABLE

forumdb=> CREATE TABLE part_tags_date_04_2023 PARTITION OF part_tags
FOR VALUES FROM ('2023-04-01') TO ('2023-04-30');
CREATE TABLE

forumdb=> CREATE INDEX on part_tags(tag);
CREATE INDEX

```

As we can see, the only two differences are `PARTITION BY RANGE` and `FOR VALUES FROM .. TO ...`

3. In this example, as in the previous example on list partitioning, we have obtained the parent table and all the child tables without complexity, and as we can see in the following snippet, the `CREATE INDEX` statement has been propagated to the child tables automatically:

```

forumdb=> \d part_tags;
           Partitioned table "forum.part_tags"
  Column  |          Type          | Collation | Nullable |
Default

```

```
-----+-----+-----+-----+-----
-----
pk      | integer          |          | not null |
nextval('part_tags_pk_seq'::regclass)
ins_date | date             |          | not null |
now()::date
tag      | character varying(255) |          | not null |

level   | integer          |          | not null | 0
Partition key: RANGE (ins_date)
Indexes:
    "part_tags_pkey" PRIMARY KEY, btree (pk, ins_date)
    "part_tags_tag_idx" btree (tag)
Number of partitions: 4 (Use \d+ to list them.)

forumdb=> \d part_tags_date_01_2023;
                                         Table "forum.part_tags_date_01_2023"
  Column |          Type          | Collation | Nullable |
Default
-----+-----+-----+-----+-----
-----
pk      | integer          |          | not null |
nextval('part_tags_pk_seq'::regclass)
ins_date | date             |          | not null |
now()::date
tag      | character varying(255) |          | not null |

level   | integer          |          | not null | 0
Partition of: part_tags FOR VALUES FROM ('2023-01-01') TO ('2023-01-31')
Indexes:
    "part_tags_date_01_2023_pkey" PRIMARY KEY, btree (pk, ins_date)
    "part_tags_date_01_2023_tag_idx" btree (tag)
```

4. As we did earlier, let's do some INSERT operations:

```

forumbd=> insert into part_tags (tag,ins_date,level) values
('Operating Systems','2023-01-01',0);
INSERT 0 1
forumbd=> insert into part_tags (tag,ins_date,level) values
('Linux','2023-02-01',1);
INSERT 0 1
forumbd=> insert into part_tags (tag,ins_date,level) values ('BSD
Unix','2023-03-01',1);
INSERT 0 1
forumbd=> insert into part_tags (tag,ins_date,level) values ('Rocky
Linux Distro','2023-04-01',2);
INSERT 0 1

```

5. And let's now check whether everything is okay:

```

forumbd=> select * from part_tags;
pk | ins_date | tag | level
----+-----+-----+-----
1 | 2023-01-01 | Operating Systems | 0
2 | 2023-02-01 | Linux | 1
3 | 2023-03-01 | BSD Unix | 1
4 | 2023-04-01 | Rocky Linux Distro | 2
(4 rows)

forumbd=> select * from part_tags_date_01_2023;
pk | ins_date | tag | level
----+-----+-----+-----
1 | 2023-01-01 | Operating Systems | 0
(1 row)

forumbd=> select * from part_tags_date_02_2023;

pk | ins_date | tag | level
----+-----+-----+-----

```